🏋 Personal Athlete Performance Platform · 2026

THE TRACKER

Track. Improve. Repeat.

- Next.js 16
- TypeScript 5
- Supabase
- Tailwind 4
- Google OAuth
- Vercel

24	0	15	11	6	3
ROUTES	API ROUTES	DB TABLES	MIGRATIONS	TRIGGERS	RBAC ROLES

PROBLEM

Why Generic Apps Don't Work

Structured athlete programs have specific requirements that mainstream fitness apps systematically fail to support.

No Rolling Cycles

Strong, Hevy, and all major apps assume a weekly schedule. An Upper/Lower/Rest rotation that repeats every 10 days is completely unsupported. Athletes must track their position in the cycle manually.

No Progression Intelligence

Generic apps log data but offer no guidance on what to do next. Athletes must recall last session weights from memory, calculate micro-progressions themselves, and decide when to increase load.

Subscription-Gated Analytics

The most useful features — PR tracking, exercise history charts, volume analytics — sit behind $10–20/month paywalls in every major fitness app. Core performance data should be free.

Slow Session Setup

Manually adding exercises and sets before every session is friction that erodes consistency. Structured athletes follow the same template each session — apps should auto-populate it.

The Solution

THE TRACKER is built around exactly one athlete's training system: Upper A → Lower A → Rest → Upper B → Lower B → Rest → Upper C → Rest → Cardio/Abs → Rest. Every feature serves this program.

FEATURES

Complete Feature Ecosystem

10 integrated features that replace 3 separate fitness apps with one fully owned platform.

Exercise Library

48 system exercises + custom. Video links, muscle groups, categories, alternatives.

Workout Templates

6 predefined sessions. Auto-populate exercises and sets on session start.

Programs

Group templates into named programs. "Upper & Lower Split" package ships by default.

Live Logger

Real-time set tracking. Progression hints. Rest timer. Swap alternatives mid-session.

PR System

Auto-detects weight, e1RM, volume records. Real-time badge. Full PR history.

Training Calendar

Month view with 10-day rolling cycle labels. Completion dots. Rest day indicators.

Weight Tracking

Daily body weight log with trend chart. Auto-syncs to body measurements via DB trigger.

Measurements

8 body circumference measurements. Waist, chest, shoulders, arms, forearms, thighs, calves.

Progress Photos

Front, side, back photo categories. Stored in Supabase Storage with RLS.

Admin Panel

RBAC-protected management of exercises, templates, alternatives, videos, users.

Dashboard

5 stats: total workouts, PRs, current weight, last session, weekly activity heatmap.

Auth & Security

Google OAuth one-click. JWT cookies. RLS on all 15 tables. 5-layer security model.

TRAINING SYSTEM

The 10-Day Rolling Cycle

THE TRACKER models a specific Upper/Lower split. The cycle repeats every 10 days, calculated using modular arithmetic from a user-configurable start date.

DAY 1	DAY 2	DAY 3	DAY 4	DAY 5
Upper A Chest · Back · Shoulders · Triceps	**Lower A** Quads · Hamstrings · Glutes · Calves	**Rest** Active recovery	**Upper B** Shoulders · Biceps · Triceps · Forearms	**Lower B** Hamstrings · Quads · Calves

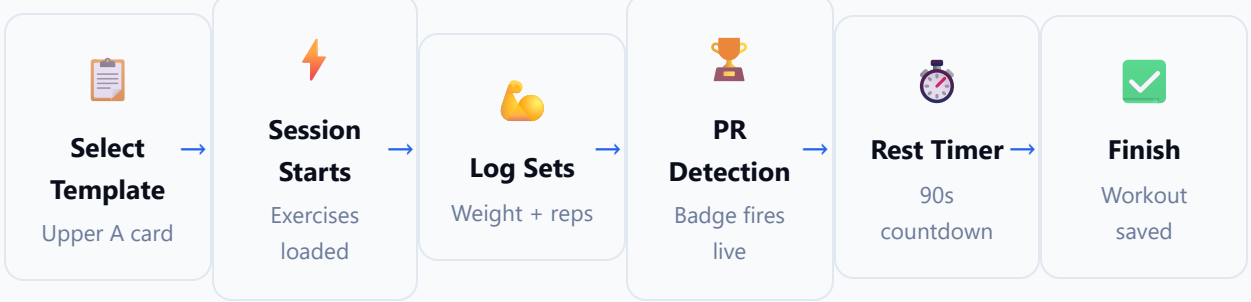
DAY 6	DAY 7	DAY 8	DAY 9	DAY 10
Rest Active recovery	**Upper C** Chest · Back · Arms · Traps	**Rest** Active recovery	**Cardio + Abs** Incline walk · Core circuit	**Rest** Active recovery

```
// Modular arithmetic — handles negative differences correctly
cycleIndex = ((differenceInCalendarDays(date, cycleStart) % 10) + 10) % 10;
currentDay = CYCLE_DAYS[cycleIndex]; // Always returns correct day 0–9
```

USER FLOW

Workout Session Flow

From template selection to workout completion — an end-to-end workout session in 6 steps.



📊 Progression Hint

Before each set, the system shows last weight and last reps. A suggested next weight is computed based on performance trend.

🔀 Exercise Swap

Up to 3 ordered alternatives per exercise. Configured by admin. Mid-session swap replaces the exercise in the active workout.

🏆 Epley 1RM

e1RM = weight × (1 + reps/30). Auto-calculated and compared to historical max. PR badge fires if new record detected.

DATABASE

Entity Relationship Overview

15 PostgreSQL tables with full RLS enforcement. All user data is isolated at the database layer — not just the application layer.

profiles

PK id	uuid
email	text UNIQUE
display_name	text
role	user|admin|super_admin
avatar_url	text

exercises

PK id	uuid
FK user_id	uuid (nullable)
name	text NOT NULL
primary_muscle_group	text
category	Upper|Lower|Cardio

workouts

PK id	uuid
FK user_id	uuid NOT NULL
FK template_id	uuid (nullable)
started_at	timestamptz
completed_at	timestamptz?

workout_entries

PK id	uuid
FK workout_id	CASCADE
FK exercise_id	RESTRICT
weight / reps	numeric, integer
set_type / is_pr	text / boolean

personal_records

PK id	uuid
FK user_id + exercise_id	UNIQUE
max_weight	numeric(6,2)
max_estimated_1rm	numeric(6,2)
max_volume	numeric(8,2)

workout_templates

PK id	uuid
FK user_id	nullable (system)
name	Upper A, Lower A…
muscle_focus	text
estimated_duration	integer (min)

Key Database Relationships

auth.users → profiles (1:1 CASCADE)

profiles → workouts → workout_entries (1:N:N — full cascade)

exercises → workout_entries (RESTRICT — cannot delete exercise with logged sets)

exercises ↔ exercises (via exercise_alternatives — self-join, max 3 ordered swaps)

weight_logs ↔ body_measurements (bi-directional sync via PostgreSQL triggers)

workout_templates → workout_template_exercises (1:N CASCADE)

SECURITY

5-Layer Security Model

Defense in depth — security enforced at every layer of the stack, from the Edge to the database.

1

Edge Middleware — proxy.ts

Validates JWT on every request. Refreshes expired tokens. Cannot be bypassed. Routes ALL matched requests.

2

Layout Auth Guard

DashboardLayout and AdministrationLayout independently verify session server-side. redirect() on failure.

3

Server Action User Validation

Every Server Action calls supabase.auth.getUser() before any DB operation. Throws on missing session.

4

Admin Role Verification — verifyAdmin()

Admin actions fetch profile.role from DB (not JWT). Cannot be faked. Throws 403 on insufficient role.

5

PostgreSQL Row-Level Security

RLS on all 15 tables. auth.uid() evaluated at query time. Even direct DB access respects user isolation. Cannot be disabled by application code.

DATABASE MIGRATIONS

Migration Chain

11 sequential SQL migrations define the complete database schema. Order matters — each migration builds on the previous.

00001 **Core Schema**

6 foundation tables: profiles, exercises, workouts, workout_entries, weight_logs, personal_records

00002 **Row-Level Security**

Enable RLS on all 6 tables. SELECT/INSERT/UPDATE/DELETE policies using auth.uid()

00003 **Database Triggers**

handle_new_user() auto-creates profile on OAuth. update_updated_at() timestamps both tables

00004 **Performance Indexes**

B-tree indexes on user_id, log_date, exercise_id, completed_at — all common filter columns

00005 **RBAC Roles**

role column (user|admin|super_admin). protect_profile_role trigger prevents unauthorized escalation

00006 **Exercise RLS**

System exercises (user_id IS NULL) visible to all. User exercises private. Enables admin curation

00007 **Exercise Fields**

category (Upper/Lower/Cardio), equipment, difficulty, alternative_exercise, notes columns

00008 **Workout Templates**

workout_templates + workout_template_exercises tables. Seeds all 6 system templates with exercises

00009 **Workout Packages**

workout_packages + junction table. Seeds "Upper & Lower Split" package with all 6 templates

00010 **Athlete Upgrade**

exercise_alternatives, pr_history, body_measurements, progress_photos, schedule_settings, bi-directional sync triggers

00011 **Storage Bucket**

Supabase Storage "progress-photos" bucket. RLS policies for authenticated upload and user-scoped access

— TECHNOLOGY STACK

Complete Tech Stack

FRONTEND

- Next.js 16.2.7 (App Router)
- React 19
- TypeScript 5
- Tailwind CSS v4
- shadcn/ui + Radix UI
- Lucide React Icons
- Recharts 3.8

BACKEND / DATA

- Supabase Auth
- PostgreSQL 15
- Supabase Storage
- Next.js Server Actions
- Google OAuth 2.0
- @supabase/ssr
- Zod Validation

DEVELOPER TOOLS

- Vercel (Deployment)
- Git + GitHub
- Turbopack (HMR)
- React Hook Form
- Sonner (Toasts)
- date-fns
- clsx + tailwind-merge

FUTURE ROADMAP

What's Next

In Progress

📏 Measurements analytics charts

📸 Before/after photo comparison slider

📽️ In-drawer YouTube video embeds

Planned

📱 PWA + offline mode (IndexedDB)

📊 Volume load analytics (weekly charts)

🔔 Push notifications for rest timer

📥 CSV / PDF data export

Future

🧠 AI-powered plateau detection

📉 Deload recommendation engine

🤝 Coach-athlete relationship mode

🌍 Multi-user expansion

PROJECT METRICS

By the Numbers

24	0	15	11	6	3
Production Routes	Custom API Routes	Database Tables	SQL Migrations	DB Triggers	RBAC Roles

6	48+	10	3	8	100%
Workout Templates	System Exercises	Cycle Days	PR Types Tracked	Body Measurements	TypeScript Build

THE TRACKER

Track. Improve. Repeat.

github.com/HusseinA-H/THE-TRACKER · Hussein A-H · 2026